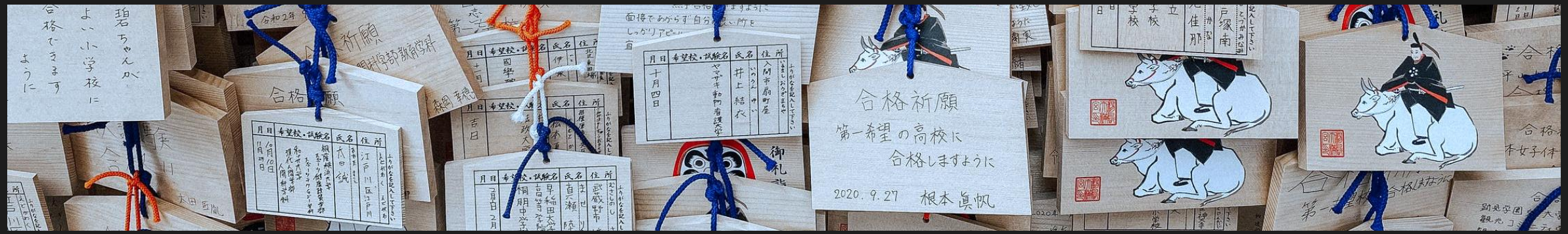
Kapitel 3

Requirements Engineering



Agenda

Abschnitt 3.1

Einführung zum Requirements Engineering

Abschnitt 3.2

Arbeitsprodukte & Dokumentation

Abschnitt 3.3

Ermittlung & Bewertung von Anforderungen

Abschnitt 3.4

Agiles Requirements Engineering

Klausurrelevante Lernziele

- Ziele des Requirements Engineering & Arten von Anforderungen nennen können
- Unterschied funktionale & nicht-funktionale Anforderung erläutern, Beispiele nennen können
- Drei der neun Prinzipien des Requirements Engineering nennen und erläutern können
- Drei Beispiele für Arbeitsprodukte nennen können
- Unterschied natürlichsprachig, vorlagen- & modellbasiert erläutern, Vor-/Nachteile nennen können
- Unterschied Lastenheft & Pflichtenheft erläutern können
- Zweck der Stakeholder-Analyse erläutern können
- Zweck des Prototyping erläutern können
- Verträglichkeit & Priorisierung von Anforderungen erläutern können
- Prinzipien agiles Requirements Engineering erläutern können
- Zweck, Aufbau & Inhalte von User Stories erläutern, Satzschablone anwenden können
- INVEST-Kriterien auf eine gegebene User Story anwenden können
- Unterschied zwischen klassischer & agiler Aufwandsschätzung erläutern können

Option: Zertifizierung

- Certified Professional for Requirements Engineering (CPRE)
- Weltweit anerkanntes Zertifikat für Requirements Engineering & Business Analyse
  - Durch: International Requirements Engineering Board (IREB)
  - Level: Foundation, Practitioner, Specialist, Expert
- Hier: Foundation Level (CPRE FL)
  - Aktuell: Version 3.2.0 (Februar 2024)
- Prüfung bei unabhängigem Zertifizierer
  - Multiple Choice, 45 Fragen, 75 Minuten, Bestehensquote: 75%, auch auf Deutsch
  - Gebühr: 275 € (Studierendenrabatt möglich)
  - Online-Prüfung möglich, Beispielprüfungen verfügbar
- Vorlesung bereitet teilweise auf die Prüfung vor

Abschnitt 3.1

Einführung zum Requirements Engineering

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Requirements Engineering

- Anforderungen: Bedürfnis nach neuen oder Anpassung bestehender Systeme
  - Funktionale Anforderungen: Struktur & Daten, Funktion & Ablauf, Zustand & Verhalten
  - Qualitätsanforderungen (NFA) [→ Kapitel 1]
  - Randbedingungen (Constraints)
- Stakeholder: alle Personen / Organisationen, die Anforderungen beeinflussen oder vom System betroffen sind
- Ziel: Ermittlung, Dokumentation, Validierung & Verwaltung von Anforderungen, so dass bereitgestellte Systeme die Bedürfnisse der Stakeholder erfüllen
- Mehrwerte für gesamte Entwicklung
  - Besseres Verständnis des Problems; Verringerung des Risikos, falsche Systeme zu entwickeln
  - Grundlage für die Schätzung des Entwicklungsaufwands
  - Voraussetzung für das Testen
- Symptome für mangelhaftes RE: fehlende / unklare / falsche Anforderungen
  - Gründe: vorschneller Beginn der Entwicklung; Kommunikationsprobleme; Annahme, dass Anforderungen selbstverständlich sind

Prinzipien I

- (1) Wertorientierung
  - Wert einer Anforderung: Nutzen abzüglich Kosten für Ermitteln, Dokumentieren, Validieren etc.
  - Nutzen einer Anforderung: Beitrag, um System zu bauen, das die Bedürfnisse der Stakeholder erfüllt und um das Risiko von Fehlschlägen & kostspieligen Nacharbeiten zu verringern
- (2) Stakeholder
  - Besitzen mindestens eine Rolle, z.B. Auftraggebende, Betreibende, Nutzende, Behörde etc.
  - Einbeziehung der richtigen Personen wichtig [→ Stakeholder-Analyse]
  - Können widersprüchliche Anforderungen habe; Konflikte müssen identifiziert & gelöst werden
- (3) Gemeinsames Verständnis
  - Dokumentierte & vereinbarte Anforderungen; problematisch: implizites Wissen
  - Positiver Einfluss: Domänenwissen; gegenseitiges Vertrauen; Glossar, Prototyp, frühes Feedback
  - Negativ: geografische Distanz; Outsourcing; große Teams; viele Fluktuationen
- (4) Kontext
  - Systeme sind immer in einem Kontext (Systemumgebung)
  - Genaue Klärung der Systemgrenze, Umfang, Schnittstellen nötig

Prinzipien II

- (5) Problem → Anforderung → Lösung
  - Alle drei eng verbunden; können nicht isoliert betrachtet werden
- (6) Validierung
  - Anforderungen müssen früh validiert werden: Einigkeit? Bedürfnisse der Stakeholder abgedeckt? Kontextannahmen vernünftig?
  - Überprüfung, ob System die tatsächlichen Bedürfnisse der Stakeholder erfüllt
- (7) Evolution
  - Systeme & Anforderungen: ständige Veränderungen, z.B. Wettbewerber, neue Technologien, Gesetzesänderungen, veränderte Bedürfnisse etc.
- (8) Innovation
  - Anstreben, um Stakeholder nicht nur zufrieden zu stellen, sondern zu begeistern
- (9) Systematische & disziplinierte Arbeit
  - Prozesse & Praktiken zur systematischen Ermittlung, Dokumentation, Validierung & Verwaltung von Anforderungen; unabhängig vom Vorgehensmodell
  - Es gibt keinen »Standardprozess«; immer Anpassung an Kontext; Erfolg laufend prüfen

Abschnitt 3.2

Arbeitsprodukte & Dokumentation

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Arbeitsprodukte

- Dokumentiertes Zwischen- bzw. Endergebnis
  - Von Skizze bis zu vertraglicher Spezifikation
  - Merkmale: Zweck, Darstellung, Umfang & Lebensdauer
  - Darstellung: natürliche Sprache; vorlagenbasiert; modellbasiert; weitere Formen, z.B. Prototyp
- Typische Arbeitsprodukte
  - Einzelne Anforderung, z.B. User Story, Software Requirements Specification (SRS)
  - Menge von Anforderungen, z.B. Anwendungsfall, grafische Modelle (z.B. UML), Schnittstellen-beschreibung, Epics, Produkt Backlog, Story Map
- Detaillierungsgrad hängt vom Zweck ab
  - Beachte: hoher Detaillierungsgrad ist aufwändig & kostspielig
  - Frühe Planung & Klärung mit Stakeholdern: Arbeitsprodukte, Darstellung, Detaillierungsgrade
- Richtlinien
  - Arbeitsprodukt passt zum Zweck; Vollständigkeit & angemessene Strukturierung; keine Redundanzen oder Inkonsistenzen; Verständlichkeit; einheitliche Fachsprache; Versionierung & Verfolgbarkeit

Natürlichsprachige Arbeitsprodukte

- Vorteile
  - Ausdrucksstärke & Flexibilität
  - Unmittelbar; keine speziellen Werkzeuge erforderlich
  - Für Auftraggebende bzw. Nutzende einfacher
- Nachteile
  - Mehrdeutigkeiten, Auslassungen, fehlende Präzision etc.
- Empfehlungen
  - Kurze & gut strukturierte Sätze
  - Definition & konsequente Verwendung einer einheitlichen Fachsprache
  - Vermeidung ungenauer / mehrdeutiger Begriffe & Phrasen
  - Kenntnis der typischen Fehlerquellen, z.B. Unvollständigkeit, Substantive, Bedingungen, passive Formulierungen, Universalquantoren, Nominalisierungen
  - Leitfaden: SOPHIST-Regelwerk, basierend auf Neurolinguistischem Programmieren (NLP)

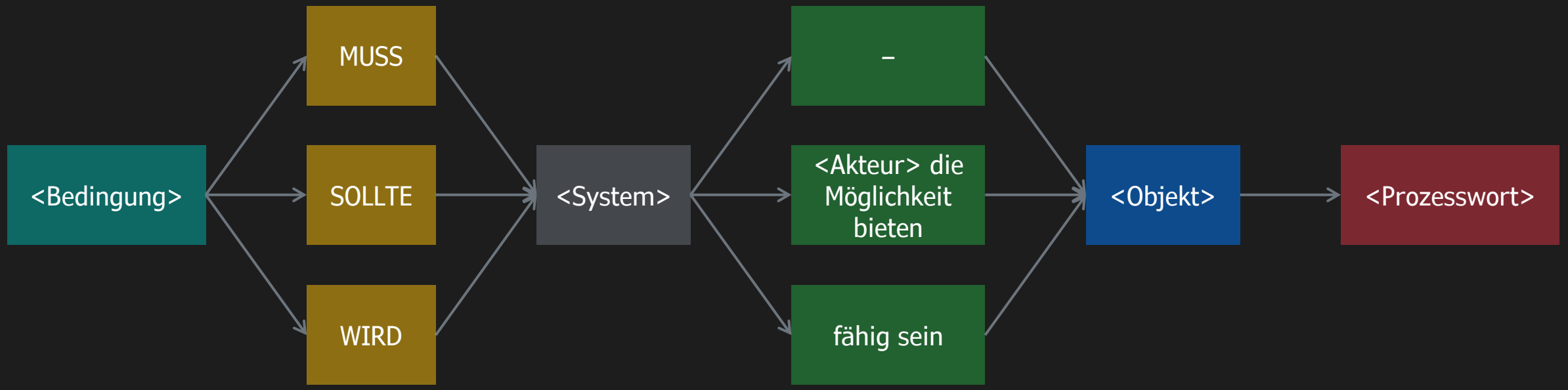
Quelle: (Rupp, SOPHIST-Gesellschaft für Innovatives Software-Engineering 2020)

Vorlagen- und modellbasierte Arbeitsprodukte

- Vordefinierten Strukturen, z.B.
  - Satzschablone: syntaktische Satzstruktur, z.B. User Story
  - Formularvorlage: bestimmte Felder, z.B. Anwendungsfall
  - Dokumentvorlage: Struktur für Anforderungsdokument, z.B. ISO 29148
  - Vorteil: einheitliche Struktur; Hilfestellung bei Erfassung wichtiger Informationen
  - Nachteil: Fokus auf Form statt Inhalt; andere Aspekte werden ggf. vergessen
- Modell: vereinfachtes, abstraktes Abbild der Realität für spezifische Aspekte
  - Oft: grafische Darstellung
  - Arten: Kontextmodelle, Modelle für Struktur & Daten, Funktion & Ablauf und Zustand & Verhalten
  - Modellierungssprachen mit standardisierter Syntax, insbesondere UML [→ Exkurs U]
  - Grenzen: nicht jede relevante Information kann ausgedrückt werden
  - Herausforderung: verschiedene Modelle integrieren & konsistent halten

Quelle: (ISO/IEC/IEEE 29148:2018)

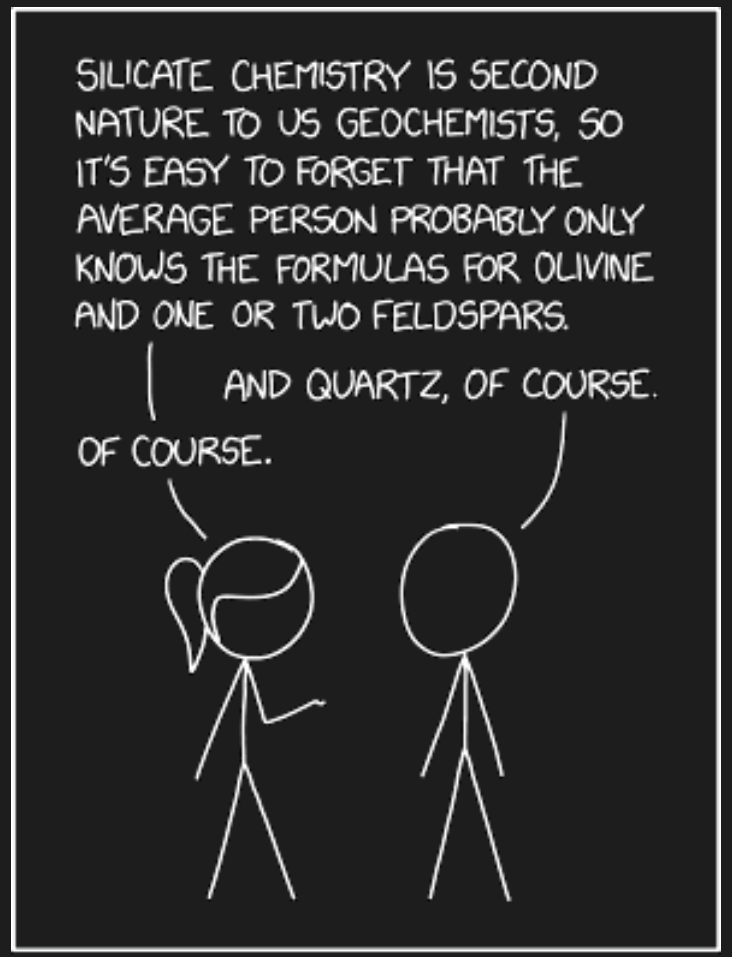
Beispiel: SOPHIST-Satzschablone



- Beispiel: »An jedem letzten Tag des Monats und unter der Bedingung, dass die Archivierung aktiviert wurde, muss das System dem Administrator die Möglichkeit bieten, die Nutzerdaten, die seit dem letzten Backup verändert wurden, auf einer Festplatte zu archivieren«

Quelle: (Rupp, SOPHIST-Gesellschaft für Innovatives Software-Engineering 2020)

Fachsprache & Fachdomäne



EVEN WHEN THEY'RE TRYING TO COMPENSATE FOR IT, EXPERTS IN ANYTHING WILDLY OVERESTIMATE THE AVERAGE PERSON'S FAMILIARITY WITH THEIR FIELD.

Quelle: Randall Munroe, https://xkcd.com/2501/

Glossar

- Sammlung von kontextspezifischen Begriffen & Abkürzungen
  - Akronym: aus Anfangsbuchstaben gebildetes Kurzwort, z.B. UML
  - Synonym: mehrere Begriffe für gleiche Dinge, z.B. Fahrstuhl / Aufzug
  - Homonym: gleicher Begriff für verschiedene Dinge, z.B. Schloss
- Ziel: gemeinsames Verständnis der Fachsprache
- Gemeinsam mit Stakeholdern erarbeiten & durchgehend pflegen
- Verwendung zur Pflicht machen; in Arbeitsprodukten korrekte Verwendung prüfen
- Beispiel:

Begriff	**Studierende Person**
Synonyme	Student, Studentin
Bedeutung	Person, die immatrikuliert ist und noch nicht exmatrikuliert wurde
Abgrenzung	Personen im Gaststudium sind ausgenommen
Gültigkeit	Mit Einschreibung entsteht neue studierende Person, die bis zur Exmatrikulation existiert
Identität	Durch Matrikelnummer eindeutig bestimmt
Unklarheiten	Namen aus anderen Schriftsystemen (z.B. Griechisch, Arabisch, Chinesisch)
Querverweise	Gaststudierende, Matrikelnummer, Immatrikulationsbescheinigung

Lastenheft & Pflichtenheft

- Lastenheft: vom Auftraggeber festgelegte Gesamtheit der Forderungen an die Lieferungen und Leistungen eines Auftragnehmers innerhalb eines Auftrages
- Pflichtenheft: vom Auftragnehmer erarbeiteten Realisierungsvorgaben aufgrund der Umsetzung des vom Auftraggeber vorgegebenen Lastenhefts
- Herausforderungen beim Lastenheft
  - Beschreibt die Anforderungen aus Sicht der Auftraggebenden in deren Fachsprache
  - Oft durch viele verschiedene Personen erstellt: Unternehmensleitung, Fachabteilungen, IT-Abteilung, Betriebs-/Personalrat, Datenschutzbeauftragte, Gleichstellungsbeauftragte etc.
  - »Nichttechnikern« fällt es oft schwer, Anforderungen formal zu spezifizieren; oft viele implizite Annahmen, Lücken, Unklarheiten & Widersprüche
  - Enthält neben funktionalen Anforderungen auch nicht-funktionale Anforderungen, Rahmen-bedingungen, Abnahmekriterien etc. ohne saubere Trennung

Quelle: (DIN 69901-5:2009)

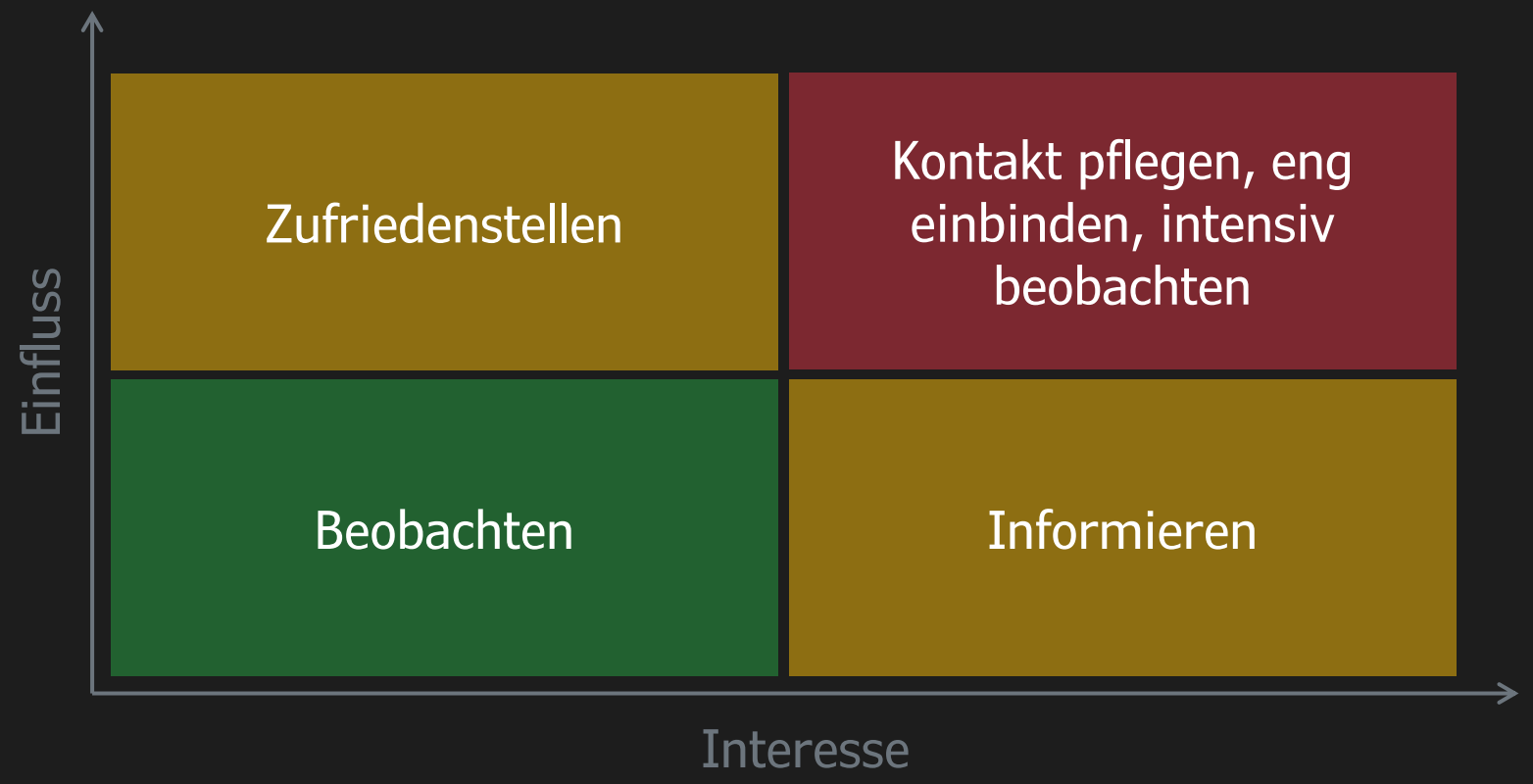
Abschnitt 3.3

Ermittlung & Bewertung von Anforderungen

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Stakeholder-Analyse

- Systematische Identifizierung aller Stakeholder mit »Interesse« und »Einfluss«
  - Erste Analyse in jedem Projekt, Teil des Stakeholder-Relationship-Management
  - Personas nutzen: fiktive Rollen als typische Stellvertretende von Nutzungsgruppen



Personas: Beispiel

Lisa

- Mitarbeiterin im Studierendensekretariat
- 38 Jahre, ausgebildete Bürokauffrau
- Arbeitet mit dem Campus-Management-System, Word, Outlook

Persönlicher Hintergrund

Lisa arbeitet seit über 12 Jahren im Studierendensekretariat. Sie mag Ihren Beruf und den Umgang mit Menschen. Sie ist geduldig, freundlich und aufgeschlossen.

Probleme & Herausforderungen

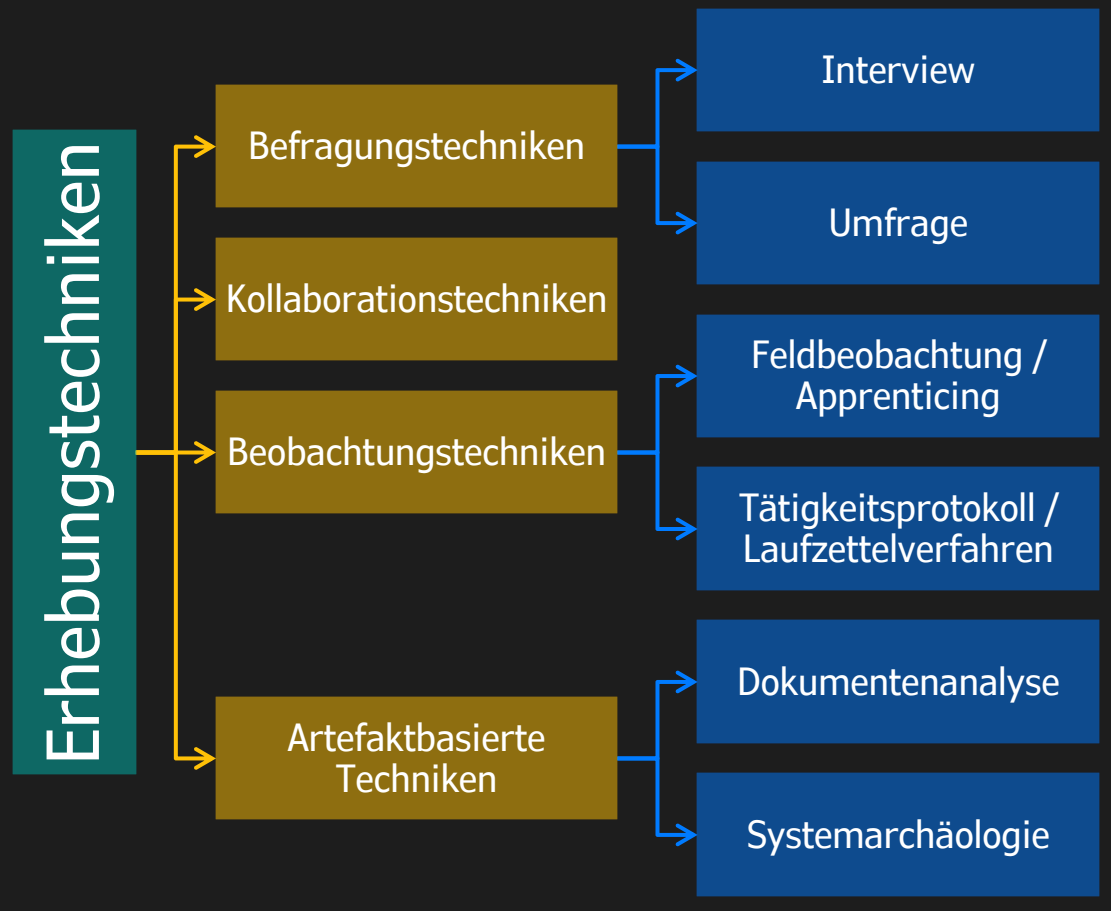
- Es ärgert Lisa, wenn häufige Funktionen nicht schnell erreichbar sind oder wenn Systemmeldungen nicht verständlich sind
- Wenn etwas nicht funktioniert, dann versucht sie selbständig, eine Lösung zu finden, z.B. durch Ausprobieren; sie guckt regelmäßig in die Online-Hilfe

Bedürfnisse

- Verständliche Meldungen und ausführliche Hilfe
- Wiedererkennung und visuelle Unterstützung
- Vertrauen, dass sie Aktionen rückgängig machen kann

Ermittlung von Anforderungen: Techniken

- Ziel: implizite Wünsche & Erwartungen in explizite Anforderungen überführen [→ Wissensmanagement]
- Beachte: es gibt nicht »die Technik«, am besten Kombination aus Techniken



Dokumentenanalyse

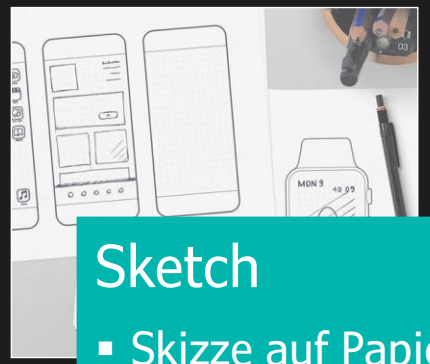
- Dokumente als Anforderungsquelle, z.B.
  - Altsysteme: Dokumentation, Handbücher, Datenmodelle, Log-Dateien
  - Geschäftsprozessdokumentation, Organisations- und Aufgabenpläne, Qualitätsmanagement-handbuch, Vordrucke & Formulare
  - Mitarbeiterhandbuch, Intranet, FAQ, Onboarding-Dokumentationen
  - Rechtliche, regulatorische und unternehmensspezifische Vorschriften
  - Marketinginformationen über potenzielle zukünftige Nutzende
- Vorteil: schnell durchführbar, wenig Störung des Betriebsablaufes
- Nachteil
  - Sehr umfangreich, wenig strukturiert
  - Ohne sachkundige Führung für Außenstehende kaum nachvollziehbar
  - Mangelnde Aktualität, Qualität, Vollständigkeit
  - Fraglich, ob Dokumente immer Ist- bzw. Soll-Zustand widerspiegeln
  - Wenig persönlicher Bezug zu Stakeholdern

Prototyping (1)

- Prototyp-Konzept aus industrieller Fertigung
  - Einzelnes Exemplar einer Serienfertigung, das »im Wesentlichen« dem Produkt entspricht
  - Prüfung & Verbesserung vor dem Aufbau der Serienfertigung
- Software-Prototyp
  - Es gibt keine Fertigung, Prototyp wäre praktisch schon Endprodukt
  - Eher »Attrappe« (aka Mockup) mit rudimentärer GUI / Funktionalität des Zielsystems
  - Stakeholdern fällt es leichter, anhand eines Prototyps Anforderungen zu präzisieren
  - Explorativer Prototyp: gemeinsames Verständnis schaffen, Anforderung klären / validieren
  - Evolutionärer Prototyp: Pilotsystem; später Ausbau zum Zielsystems [→ MVP]

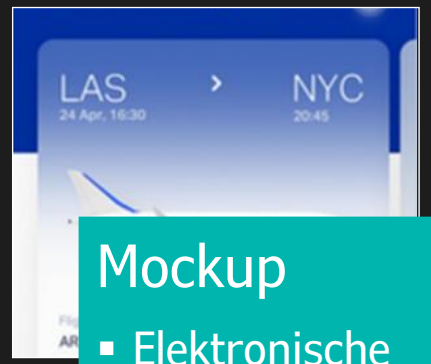
> Rapid Prototyping – Type of prototyping in which emphasis is placed on developing prototypes early in the development process to permit early feedback and analysis in support of the development process.
>
> (ISO/IEC/IEEE 24765:2017)

Prototyping (2)

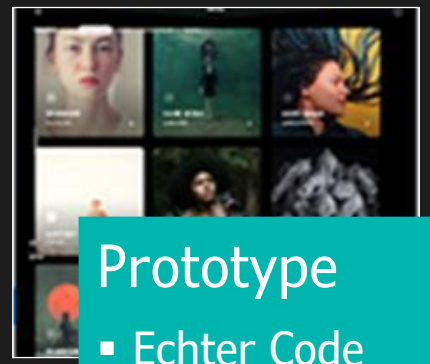


Sketch
- Skizze auf Papier

Wireframe
- Gestaltungstudie
- »Designskelett«



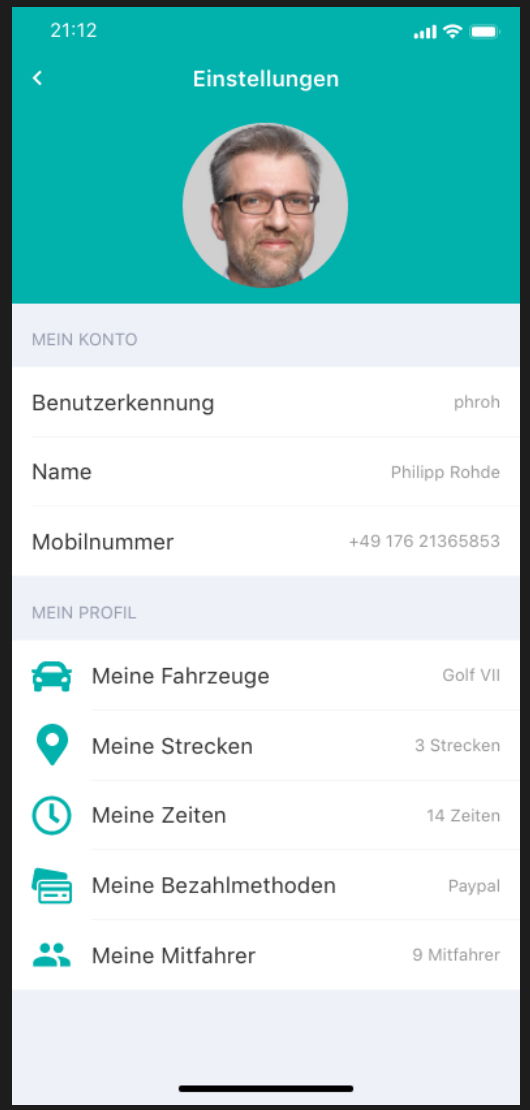
Mockup
- Elektronische Attrappe
- Look & Feel
- Simulationen

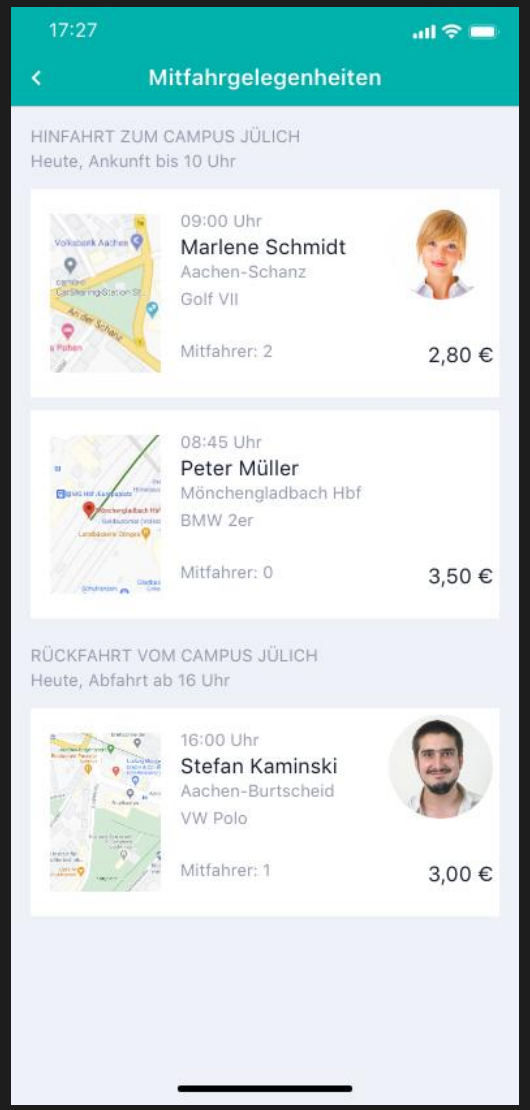


Prototype
- Echter Code
- Teile der Funktionalität

- Prototyp ist mehr als nur ein »statisches Bild«
- Unterstützt Anforderungsspezifikation, kann diese aber nicht ersetzen
- Gefahren beim Prototyping
  - Prototypen können sehr zeit- und kostenintensiv werden; vorher genaue Fragestellung festlegen
  - Prototyp ist »Wegwerf-Produkt«, aber aufgrund von Zeitmangel wird manchmal ein (reifer) Prototyp zum (unreifen) Produkt ausgebaut
- Werkzeuge: z.B. Axure RP, Mockplus, Balsamiq, Proto.io, Marvel

Prototyping: Fiktive Webanwendung





Erstellt mit Proto.io (https://proto.io)

Klassifizierung von Anforderungen

Noriaki Kano
(* 1940)

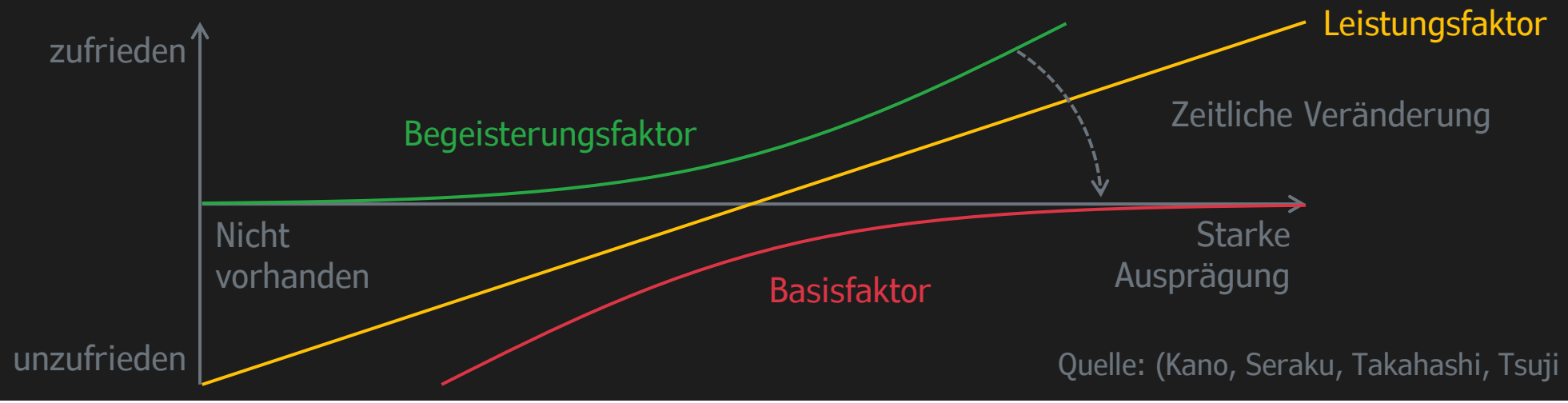
- Kano-Modell: Zufriedenheit von Nutzenden

Basisfaktor (Dissatisfier)
- Selbstverständlich
- Negativ bei Fehlen
- Nicht ausgesprochen (implizit)
- Unterbewusstes Wissen

Leistungsfaktor (Satisfier)
- Spezifiziert
- Kalkuliert: positiv / negativ
- Ausgesprochen (explizit)
- Bewusstes Wissen

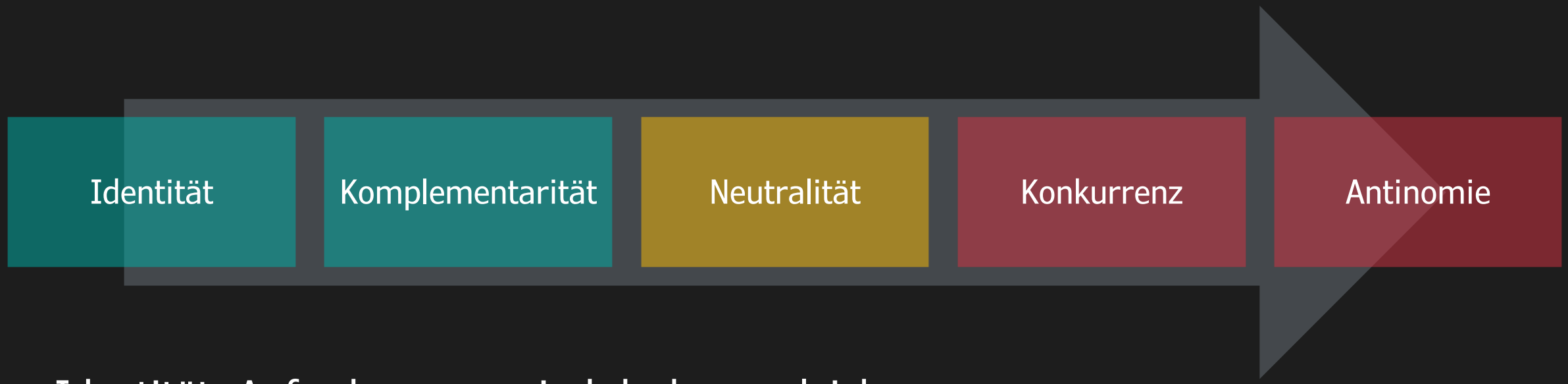
Begeisterungsfaktor (Delighter)
- Nicht erwartet
- Positiv bei Entdeckung
- Unbewusstes Wissen

- Klassifizierung ist subjektiv & kann sich ändern (Gewöhnungseffekt)
- Kano-Analyse ist Teil des User Experience Design



Quelle: (Kano, Seraku, Takahashi, Tsuji 1984)

Verträglichkeit von Anforderungen



- Identität: Anforderungen sind deckungsgleich
- Komplementarität: Umsetzung einer Anforderung führt zur (Teil-)Umsetzung der anderen Anforderungen
- Neutralität: Umsetzungen der Anforderungen sind unabhängig voneinander
- Konkurrenz: Umsetzung beider Anforderungen kann nicht gleichzeitig im selben Umfang erfolgen
- Antinomie: Anforderungen schließen sich gegenseitig aus

Priorisierung von Anforderungen

- Bewertungskriterien, z.B. Geschäftswert, Dringlichkeit, Aufwand
- Beispiel: MoSCoW-Priorisierung

Must	Should	Could	Won't
▪ Umsetzung unbedingt erforderlich	▪ Sollte umgesetzt werden, wenn alle Must trotzdem erfüllt werden können	▪ Kann umgesetzt werden, wenn Erfüllung von Must und Should nicht beeinträchtigt wird	▪ Wird (zunächst) nicht umgesetzt, aber für die Zukunft vorgemerkt

- Schritte zur Priorisierung
  - Ziele & Rahmenbedingungen festlegen, Beurteilungskriterien definieren
  - Welche Stakeholder müssen einbezogen werden?
  - Welche Anforderungen müssen priorisiert werden?
  - Auswahl der Priorisierungstechnik & Durchführung

Priorisierung: Präferenzmatrix

	Anforderung	B	C	D	E	F	Häufigkeit	Rang
A	Anforderung #1	A	C	A	A	A	4	**2**
B	Anforderung #2	–	C	D	E	F	0	**6**
C	Anforderung #3	–	–	C	C	C	5	**1**
D	Anforderung #4	–	–	–	E	F	1	**5**
E	Anforderung #5	–	–	–	–	F	2	**4**
F	Anforderung #6	–	–	–	–	–	3	**3**

- Vorgehen
  - Zu vergleichende Anforderungen auswählen
  - Paarweiser Vergleich, z.B. »Ist die Umsetzung von Anforderung #1 (A) wichtiger als Anforderung #2 (B)?« Hier: Ja, daher »A« in Spalte »B«
  - Häufigkeiten der Nennungen zählen
  - Rangfolge ergibt sich aus den Häufigkeiten

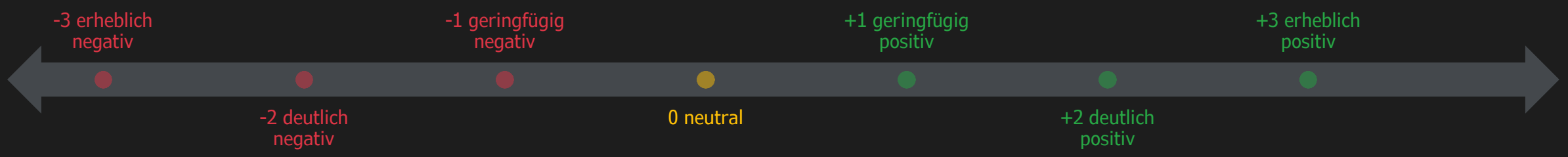
Nutzwertanalyse

- Entscheidungstheorie, auch Punktwertverfahren
- Multidimensionalen Bewertung und Auswahl von Alternativen, insbesondere bei »weichen« Bewertungskriterien
- Vorgehen
  - Alternativen $A_1, \dots, A_n$ und Bewertungskriterien $K_1, \dots, K_m$ festlegen
  - Gewichtung w_j des Kriteriums K_j ermitteln, z.B. Werte zwischen 1 und 3
  - Bewertung v_{ij} der Alternative A_i bzgl. Kriterium K_j mit fester Skala, z.B. von -3 (»erheblich negativ«) bis $+3$ (»erheblich positiv«)
  - Bewertungskoeffizient v_i von A_i

$$v_i = \frac{\sum_{j=1}^{m} w_j \cdot v_{ij}}{\sum_{j=1}^{m} w_j}$$

  - Wähle Alternative A_i mit höchstem Bewertungskoeffizient v_i
- Sollte durch mehrere Stakeholder aus unterschiedlichen Interessengruppen durchgeführt werden; Mittelwerte aller Teilnehmer einer Interessengruppe

Nutzwertanalyse: Beispiel für drei alternative Produkte



Kriterium	Gewichtung	Bewertung A_1	Bewertung A_2	Bewertung A_3
Bedienungsfreundlichkeit	3	$+3$	0	-1
Geschwindigkeit	2	$+2$	$+2$	$+3$
Sicherheit	3	-1	$+1$	$+3$
Wartungsaufwand	1	-1	$+2$	-2
Flexibilität	1	0	-1	1
Koeffizient	$\mathbf{10}$	$\mathbf{0,9}$	$\mathbf{0,8}$	$\mathbf{1,1}$

Wähle dieses Produkt

Management von Anforderungen

- Frühe & regelmäßige Prüfung der Gesamtheit aller Anforderungen auf Qualität, insbesondere Vollständigkeit, Verträglichkeit & Konsistenz
- Techniken
  - Reviews, z.B. Walkthroughs und Inspektionen
  - Exploration, z.B. Prototyping, A/B-Tests, MVP
  - Aufwand & Formalität hängt ab vom Vorgehensmodell, Reifegrad der Entwicklung, Komplexität & Kritikalität des Systems, gesetzliche & regulatorische Rahmenbedingungen etc.
- Anforderungen selten statisch: Änderungen müssen richtig gehandhabt werden
  - Beispiel: formeller Änderungsantrag (Request for Change, RfC), Eintrag im Product Backlog etc.
  - Klassische Modelle: formelle Entscheidung durch Change Control Board (CCB)
  - Zusammenhang zu Change Management; Best Practice gemäß ITIL 4
- Requirements Management
  - Prozess der Verwaltung aller Anforderungen, die in Arbeitsprodukten dokumentiert sind
  - Benötigt Konfigurationsmanagement, Versionsverwaltung & Rückverfolgbarkeit [→ Kapitel 2]
  - Transparente Verwaltung des Lebenszyklus jeder Anforderung
  - Sinnvoll: Einsatz von geeigneten Werkzeugen

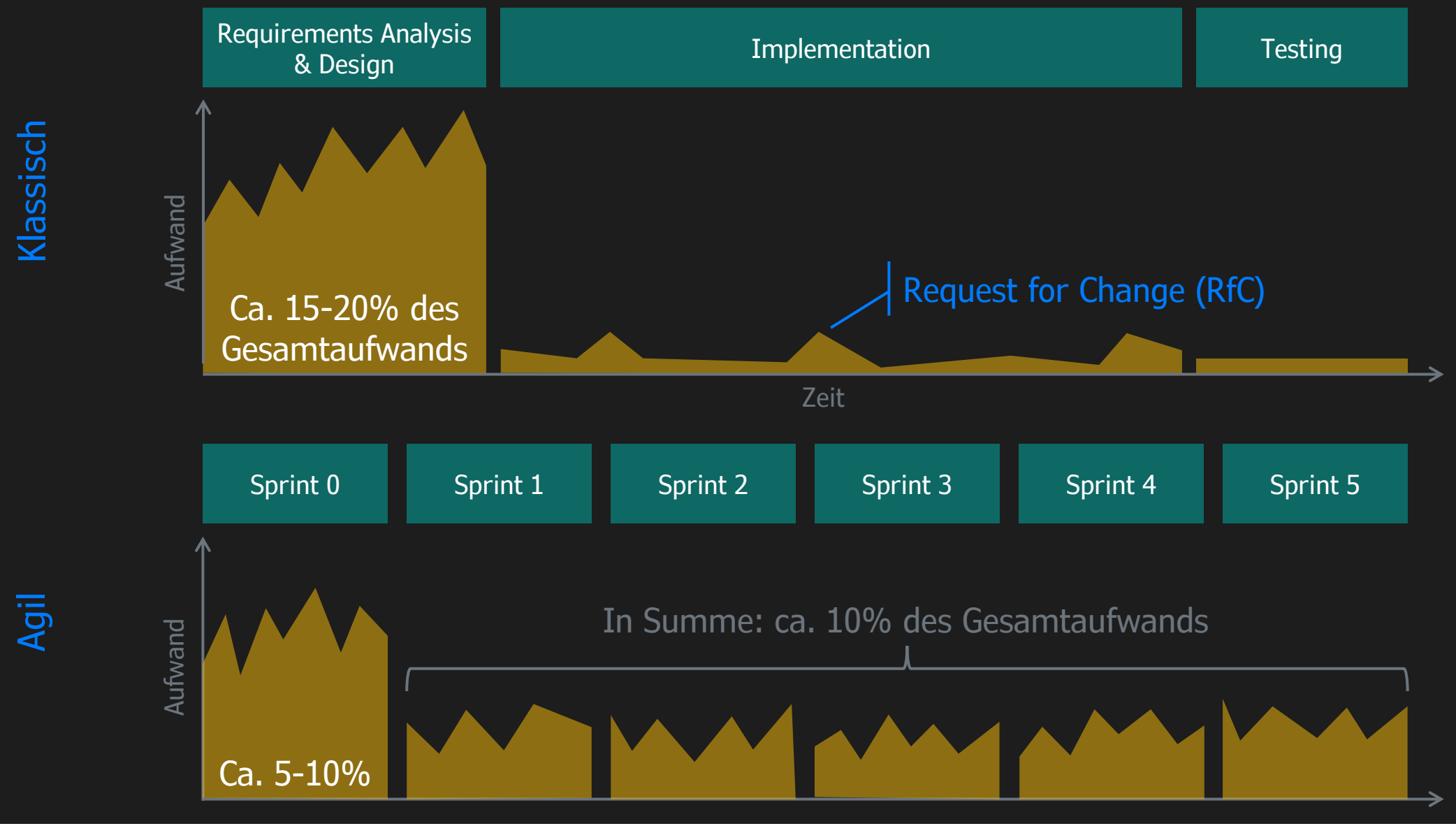
Quelle: (AXELOS 2019)

Abschnitt 3.4

Agiles Requirements Engineering

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Klassisches vs. agiles Requirements Engineering



Agiles Requirements Engineering: Prinzipien

- Frühe Beteiligung der Entwickelnden, schon im Requirements Engineering
- Feinspezifikation
  - So spät wie möglich; Detaillierung erst kurz vor der Implementierung
  - Details möglichst in Form von Testfällen
  - Keine Details zur möglichen Umsetzung
- Änderungen jederzeit zulassen
- Dokumentation & Verfolgbarkeit so effizient wie möglich
- »Königsweg« für Ermittlung von Anforderungen gibt es nicht
  - Jedes Arbeitsprodukt, das das »Gesamtbild« klärt, ist erlaubt
  - Jede Methode, die derartige Arbeitsprodukte produziert, ist erlaubt
- Ziel: Anforderungskatalog, der richtig, vollständig, konsistent, verständlich & priorisiert ist [→ Qualitätsmerkmale im Requirements Engineering]

Visions, Goals & Epics

- Eine gemeinsame Vision (für »Elevator Pitch«)
- Einige Goals: funktional, organisatorisch, zeitlich, finanziell oder qualitativ
- Epics
  - Übergeordnete oder komplexe User Story; Sammlung von User Stories; Ideen mit noch ungeklärten Details etc.
  - Umsetzung ist in einem Sprint (vermutlich) nicht möglich
  - Sind priorisiert, aber nur bedingt testbar
  - Arten: Business Epic, Architectural Epic, Non-Functional Epic

Ziel (zeitlich): »Bis Ende des Jahres ist die neue Lösung für alle gängigen Browser verfügbar.«

Epic: »Der elektronische Übungsbetrieb ist mit dem zentralen Student Information System integriert.«

User Stories

User Story – (1) Simple narrative illustrating the user goals that a software function will satisfy.

(2) A narrative description of a software requirement, function, feature, or quality attribute, presented as a narrative of desired user interactions with a software system.

(ISO/IEC/IEEE 24765:2017)

- Wichtigstes Arbeitsprodukt: Funktionalität, die für Nutzende von Wert ist / Problem löst / Prozess unterstützt
- Neben Beschreibung weitere Pflichtangaben
  - Spezifische Akzeptanzkriterien
  - Komplexität bzw. Aufwand (typisch: Story Points)
- Detaillierungsgrad reicht für Planung & Aufwandschätzung des nächsten Sprints (gemäß Priorität im Product Backlog)
- Wichtig: User Story kann (voraussichtlich) in einem Sprint umgesetzt werden

User Stories: Satzschablone

Als <Rolle> möchte ich <Aktion>, damit <Ziel/Nutzen>.

- Rolle sollte möglichst konkret sein, nicht nur »Nutzende«
- Beispiele

»Als teilnehmende Person möchte ich über neue Übungsblätter benachrichtigt werden, damit ich kein Übungsblatt verpasse, auch wenn ich nicht regelmäßig im System nachschaue.«

»Als korrigierende Person möchte ich die Summe der erreichten Übungspunkte jeder teilnehmenden Person einsehen können, damit ich sie gezielt unterstützen kann.«

User Stories: Akzeptanzkriterien

- Was muss funktional gegeben sein, damit die User Story als abgeschlossen gilt?
- Akzeptanzkriterien sind stichwortartig, nicht zu detailliert
- Keine genaue Test-Spezifikationen, aber Grundlage dafür
- Beispiel

»Als korrigierende Person möchte ich die Summe der erreichten Übungspunkte jeder teilnehmenden Person einsehen können, damit ich sie gezielt unterstützen kann.«

Akzeptanzkriterien:

- Auswahlmöglichkeit der Teilnehmenden, die der korrigierenden Person zugeordnet sind
- Anzeige der erreichten Übungspunkte, maximale Übungspunkte, Prozentwert
- Diagramm des Punkteverlaufs in vergangenen Übungsblättern
- Grafischer Indikator, wenn Prozentwert unterhalb eines global konfigurierbaren Wertes

Story Card

USER STORY 96

96 My first user story

Philipp Rohde | 0 comments | Add tag | Save | Follow

State	Active	Area	Blue Viper
Reason	Implementatio...	Iteration	Blue Viper\Sprint 0

Updated by Philipp Rohde: 3m ago

Details | (4)

Description

As a <role> I can <capability>, so that <receive benefit>.

Acceptance Criteria

Notes about what the story must do in order for the product owner to accept it as complete (see Mike Cohn, 2019)

Discussion

Add a comment. Use # to link a work item, ! to link a pull request, or @ to mention a person.

Planning

Story Points
3

Priority
2

Risk

Classification

Value area
Business

Deployment

Development

Related Work

Abgrenzung von User Stories

- User Story ist immer funktional
  - Nicht-funktionale Anforderungen sind Operation Stories (oder andere Formate)
  - Falsch: »Als korrigierende Person möchte ich, dass bis zu 25 Teilnehmende gleichzeitig Dokumente hochladen können, damit in Spitzenzeiten keine Engpässe bestehen.«
- User Story enthält keine Aussagen zu Technik oder Umsetzung
  - Das sind Developer Stories
  - Falsch: »Als korrigierende Person möchte ich, dass meine Teilnehmenden in einer separaten Datenbank-Tabelle erfasst werden, damit es keine Überschneidungen gibt.«
- User Story versus Use Case
  - Use Case: Fokus auf Abläufe bzw. sichtbares Verhalten
  - User Story: Fokus auf Funktion
  - Vollständiger Use Case ist im Allgemeinen wesentlich detaillierter
  - Use Cases können genutzt werden, um Details und Ablauf von User Stories weiter zu präzisieren

Qualitätskriterien für User Stories

- INVEST-Kriterien von Bill Wake

Independent

- User Stories sind möglichst unabhängig voneinander

Negotiable

- User Stories können geändert, zerlegt oder zusammengeführt werden

Valuable

- Jeder User Story hat erkennbaren Wert für Nutzende

Estimatable

- Aufwand jeder User Story ist abschätzbar

Small

- Jede User Story ist in einem Sprint umsetzbar

Testable

- Die korrekte Umsetzung jeder User Story kann eindeutig geprüft werden

Quelle: (Wake 2003)

User Stories: schlechte Beispiele

»Als Benutzer möchte ich eine Übersicht der Ergebnisse ausdrucken können, damit ...«

Zu unspezifisch!
Verstoß: N, E, S, T

»Als korrigierende Person möchte ich die Punkte in Rot angezeigt bekommen, wenn sie weniger als 25% sind; in Gelb, wenn sie ...«

Enthält Umsetzungsdetails!
Verstoß: N

»Als lehrende Person möchte ich nach bestimmten Übungen filtern können.«

Grund / Wert für Nutzende?
Verstoß: V

»Als korrigierende Person möchte ich Gruppen bilden können, damit ich diese benachrichtigen, gemeinsame Übungspunkte vergeben und die Liste ausdrucken oder per Mail versenden kann.«

Zu umfangreich!
Verstoß: N, S, ggf. auch I

Wann ist eine User Story im Sprint abgeschlossen?

Developer

- Alle zugehörenden Tasks sind erledigt

Product Owner

- Die spezifischen Akzeptanzkriterien sind erfüllt

Organisation

- Die allgemeinen Bedingungen der Definition of Done sind erfüllt

- Letztlich entscheidet immer der Product Owner im Sprint Review, ob einer User Story tatsächlich abgeschlossen wurde

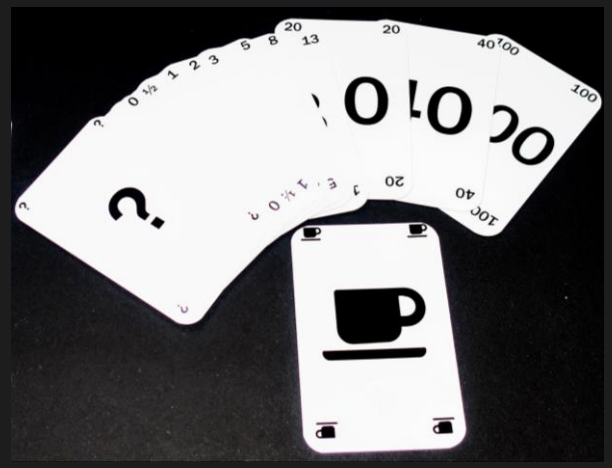
Weitere Arbeitsprodukte

- Constraint
  - Übergreifende, nicht-funktionale Anforderung oder technische Rahmenbedingung, z.B. Qualitätsmerkmale, Frameworks, Usability, Corporate Design, gesetzliche Vorgaben etc.
- Spike
  - Interne Forschungsaufgabe, Experiment, Sandbox, Analyse etc. bei Unklarheiten von Anforderungen, Aufwand, Umsetzung etc.
- Development Story, Test Story, Operation Story
  - Spezifische Story für Entwicklung, Test oder Betrieb [→ DevOps]
- Impediment Story (aka Team Story)
  - Optimierung der Zusammenarbeit
  - Ergebnis aus Scrum Retrospective

Aufwandsschätzung

- Aufwandsschätzung bei klassischen Vorgehensmodellen
  - Schätzung erfolgt früh & mit viel Unschärfe; Umfang, Kosten, Lieferzeitpunkt werden fixiert
  - Auftraggebende & Entwickelnde lernen erst im Projekt die genauen Anforderungen
  - Folge: Termine & Kosten werden nicht gehalten, Anforderungen nur teilweise umgesetzt
- Agiler Softwareentwicklung
  - Genauer Funktionsumfang bleibt variabel
  - Frühe Grobschätzung zur Priorisierung, Detaillierung erst kurz vor Implementierung
  - Relative Schätzungen, z.B. Story Points statt Personentagen
- Vorgehen setzt hohes Vertrauen auf beiden Seiten voraus
  - Auftraggebende vertrauen, dass ein Softwaresystem geliefert wird, das die Probleme löst
  - Auftragnehmende vertrauen, dass nicht ständig neue Funktionalität eingefordert wird (»Moving Targets«)

Story Points & Planning Poker



- Story Points: abstrakte Größe, keine Zeiteinheit!
  - Fibonacci-Folge: $0, \frac{1}{2}, 1, 2, 3, 5, 8, 13, 20, 40, 100, ?, \infty$
  - Alternativ: T-Shirt-Größen S, M, L, XL, XXL
  - Finde Referenzstory mit »mittlerer« Komplexität: Wert 3 oder 5
  - Team schätzt jede User Story relativ zur Referenzstory
- Velocity des Entwicklungsteams
  - Story Points, die das Entwicklungsteam in einem Sprint umsetzen kann
  - Geschätzter Erfahrungswert (vorher) oder tatsächlicher, gemittelter Wert (nachher)
- Planning Poker (time-boxed, z.B. 5 Minuten/User Story)
  - User Story wird erläutert & Fragen geklärt
  - Jeder wählt verdeckt einen Schätzwert (Poker-Karte); Karten werden gleichzeitig offen gelegt
  - Starke Abweichung: Team diskutiert Extremwerte & wiederholt Schätzung (max. drei Runden)
- Von Story Points zu Kosten
  - Methode 1: Aufwand für Referenzstory klassisch abschätzen, andere Stories in Relation dazu
  - Methode 2: Berechnung gemäß Kosten des Sprints & Velocity des Teams